USER_GUIDE

Action-Prefix System — User Guide

Field	Value
Revision	
Created	- -
Last modified	- - T : : Z
Status	active
Scope	end-user guide to the universal `ACTION_NAME ::` prompt-prefix system (§ . .)
Audience	end users of any CLI agent that includes the constitution submodule
Inputs	`docs/research/action_prefix_system/{DESIGN.md,GRAMMAR_ADDENDUM.md,IMPLEMENTATION_REPORT.md}`

> Anti-bluff (§ . .): this guide documents only what the design + the verified implementation actually do. Where a feature is designed but not yet implemented, that is stated explicitly as **(spec-only)** — never implied to work. The honest § . . per-agent support boundary is in § .

. What the action-prefix system is

The action-prefix system lets you put a short, UPPERCASE **action keyword** at the very start of your prompt to apply a registered behaviour to the rest of it — without retyping a long instruction every time.

When your prompt's first non-blank line starts with an action keyword followed by `::` (one space, two colons, one space), the agent looks the keyword up in a shared **action registry**, replaces the `ACTION_NAME ::` prefix with that action's full **expansion text**, and then executes the rest of your prompt under that expanded instruction.

Example. You type:

```
BACKGROUND :: Investigate the audio-routing regression on D3 and propose a fix.
```

The agent expands `BACKGROUND ::` into the registered BACKGROUND instruction (run as a background, subagent-driven, parallel work stream that produces rock-solid physical proof — see §) and then performs your actual task (*"Investigate the audio-routing regression on D and propose a fix"*) under that instruction.

The system is **universal** (works across every supported CLI agent), **extensible** (new actions are added by editing one data file, never code — see the Manual), and **loads out-of-the-box** once the constitution submodule is included in a project (see §).

Canonical authority: constitution submodule § . . (`Constitution.md`).

. The five equivalent invocation forms

Every registered action is **designed** to be invocable in five equivalent forms — same action, same expansion, same execution. The first non-blank line of your prompt is what is matched.

	Form	Example	Notes	Status
	`ACTION_NAME :: <rest>`	`BACKGROUND :: Do X`	bare `::` form (no namespace)	**Implemented**
	`PREFIX::ACTION_NAME :: <rest>`	`DEFAULT::BACKGROUND :: Do X`	namespaced `::` form	**Implemented** (grammar / ; see §)
	`/ACTION_NAME <rest>`	`/BACKGROUND Do X`	bare slash form — ONLY if `/ACTION_NAME` does NOT collide with an existing/built-in slash command of the host agent	**Implemented** (generated slash command; see §)
	`/PREFIX::ACTION_NAME <rest>`	`/DEFAULT::BACKGROUND Do X`	namespaced slash form — disambiguates from any existing `/ACTION_NAME`; ALWAYS safe	**Implemented** (hook E E; see §)

	Form	Example	Notes	Status
	`ACTION_NAME ---> <rest>` (and `PREFIX::ACTION_NAME ---> <rest>`)	`BACKGROUND ---> Do X` / `DEFAULT::BACKGROUND ---> Do X`	arrow form — the `--->` body separator (one space each side); a third equivalent delimiter alongside `::` and `/`	**Implemented** (grammar / ; see §)

These five forms are equivalent by design:

```
BACKGROUND :: ≡ DEFAULT::BACKGROUND :: ≡ /BACKGROUND ≡ /DEFAULT::BACKGROUND ≡ BACKGROUND ---> ≡ DEFAULT::BACKGROUND --->
```

> **Honest status note (§ . .).** All five forms are implemented today and work end-to-end. Forms **and** (the `DEFAULT::` namespace) and form (the arrow `--->`) are wired into the registry, the expander library, the parse paths (python + awk, byte-identical), and the LAYER- hook, and are verified (grammar suite / , paired-mutation meta-test /) — see § . Any of the five forms can be used today.

. The separators (do not confuse them)

- **Namespace separator** — `::` with **no surrounding spaces**, used INSIDE the token: `PREFIX::ACTION_NAME` (forms , , and the namespaced arrow).

- **Action-body separator** — `::` with **a space on each side**, used between the token and the rest of your task in the `::` forms (forms and).
- **Arrow-body separator** — `--->` (space, three hyphens, greater-than, space) with **a space on each side**, used between the token and the rest in the arrow form (form): `BACKGROUND ---> Do X`.
- **Slash-form body separator** — a single space, between the slashed token and the rest (forms and): `/BACKGROUND Do X`.

The exact `::` and `--->` separators are chosen deliberately so the prefix never collides with C++ `Foo::Bar`, YAML `key: value`, URLs, `12::34`-style typos, or a `-->` / `->` used in ordinary prose. An arrow that sits mid-line after a `::` token (e.g. `A :: B ---> C`) parses as the `::` form, not the arrow form.

. The `BACKGROUND` action's effect

`BACKGROUND` is the first registered action. Its expansion text (verbatim from the registry) is:

> *"The following prompt that we will provide MUST BE executed in background in parallel with all main work streams using the subagents-driven development approach! All work done MUST PRODUCE rock solid evidence covered with hard physical proof(s) that all done is working as expected and as specified without any false results and without any bluff!"*

In plain terms, prefixing your task with `BACKGROUND ::` tells the agent to:

- run the task as a **background work stream** in parallel with other work,
- drive it via the **subagent-driven** approach,
- and produce **rock-solid physical proof** that the result really works — no false results, no bluff.

It composes with the constitution's subagent-driven (§ . . / § . .), parallel-stream (§ . . / § . .), background-execution (§ . .), and captured-evidence (§ . . / § . . / § . .) + anti-bluff (§ .) anchors.

. The `REMINDER` action's effect

`REMINDER` is the second registered action. Use it to **re-surface work you scheduled or requested earlier** that is critical and whose status you are no longer sure of — for example: `REMINDER ---> the D3 audio-routing fix we queued`. Its expansion tells the agent to **never assume** the work is done or not done (a false "already done" is a bluff, a false "not started" wastes effort). Instead the agent:

- **FIRST verifies the ACTUAL current status** from captured evidence — git log/state, the task list, running and queued background work (including the BACKGROUND durable queue at `docs/requests/background_queue.md`), test artifacts, the workable-items DB + docs, and the recording corpus;
- **THEN acts on the delta** — reports the captured proof if the work is genuinely complete, resumes from the exact point if it is partial, actions it NOW with high priority if it was not started, or surfaces the block (§ . . / § . .) if it is blocked;

- and **always produces a status verdict** (done+proof / resuming-from-X / starting-now / blocked-because-Y) — it never silently no-ops.

It composes with no-guessing (§ . .), the endless-loop / zero-idle / parallel routine (§ . . / § . . / § . . / § . .), the four-layer "done" verification (§ . .), validate-just-fixed-first (§ . .), and the crashed/lost-work-never-forgotten registry (§ . .).

More actions can be added later; each gets its own keyword, expansion text, and optional rules. See the Manual §"How to add a new action".

. The escape — discussing an action without triggering it

If you want to *write about* an action keyword without triggering its expansion (for example, to ask a question about it), put a single backslash `\` at the very start of the line:

```
\BACKGROUND :: x
```

The agent strips the leading backslash and treats the line as the ordinary prompt `BACKGROUND :: x` — **no expansion happens.** The same escape applies to the arrow form (`\BACKGROUND ---> x`) and the slash forms (`\/BACKGROUND x`).

. Other grammar rules you should know

These rules keep the system predictable and prevent accidental triggers:

. **Anchored at the first non-blank line only.** The keyword must be the very start of your prompt's first non-blank line. `please BACKGROUND :: x` does **not** trigger — the keyword is not at line start, so it is an ordinary prompt. Leading blank lines before the keyword are allowed.

. **UPPERCASE only.** Action keywords are `[A-Z][A-Z0-9_]*`. `background :: x` (lowercase) does **not** trigger — this makes the keyword visually unmistakable and avoids matching ordinary prose.

. **Exact separator.** The `::` forms require exactly `::` (one space each side). `BACKGROUND::do X` (no spaces) does **not** trigger.

. **Stacked keywords.** `OUTER :: INNER :: do X` applies outer-to-inner, left-to-right: expand `OUTER`, re-scan the remainder, expand `INNER`, then `do X` is the task. Both expansions apply. *(In practice only `BACKGROUND` is registered today, so stacking is available but rarely used until more actions land.)*

. **Unknown keyword — never guessed.** If your first-line token looks like an action (`SOMETHING :: …`) but is **not** in the registry, the agent will **not** silently expand it or silently drop it. It asks which registered action you meant (per § . . / § . .), or treats your line literally — it never invents an expansion (§ . .). So a typo like `BACKROUND :: x` gets a clarifying question naming the closest registered action, not a guess.

. **No-op when nothing matches.** Any prompt that does not satisfy the grammar is just an ordinary prompt; the system does nothing.

. Per-agent support matrix

The system is built in two layers (full architecture in the Manual):

- **LAYER (always-on, every agent):** a recognition instruction is embedded in every agent context carrier (`CLAUDE.md` , `AGENTS.md` , `QWEN.md` , `GEMINI.md`). The agent reads it on every prompt and applies the free-form prefix **itself**. This is the universal floor — it works on every agent with zero extra setup.
- **LAYER (mechanical, where the agent exposes a pre-submit seam):** on Claude Code a `UserPromptSubmit` hook rewrites/injects the expansion deterministically before the model sees the prompt; on Gemini/Qwen/Codex a generated **slash command** (`/background`) is the mechanical convenience.

Agent	Free-form `BACKGROUND :: …` (form)	Mechanical interception	Slash command (form)
Claude Code	LAYER + LAYER hook (transparent)	**YES** — `UserPromptSubmit` hook rewrites before the model sees it	(hook covers it)
Gemini CLI	LAYER (agent self-applies)	NO transparent rewrite	`/background` (generated `.gemini/commands/background.toml`)
Qwen Code	LAYER (agent self-applies)	NO transparent rewrite	`/background` (generated `.qwen/commands/background.toml`)
OpenAI Codex CLI	LAYER via `AGENTS.md`	NO transparent rewrite	`/prompts:background` (generated `prompts/background.md`)
GitHub Copilot CLI	LAYER via `AGENTS.md` + `copilot-instructions.md`	NO	(custom agent optional)
Cursor	LAYER via `.cursor/rules` + `AGENTS.md`	NO	—
Aider	LAYER via `CONVENTIONS.md` / `AGENTS.md`	NO	—

Agent	Free-form `BACKGROUND :: …` (form)	Mechanical interception	Slash command (form)
Cline / Continue / Roo	LAYER via `.clinerules` / `.continue/rules` / `AGENTS.md`	NO	—

. Honest § . . boundary — Claude-Code-only mechanical rewrite

Transparent, mechanical free-form interception — the `BACKGROUND :: …` string being rewritten *before the model ever sees it* — is genuinely available **ONLY on Claude Code**, because Claude Code is the only agent (verified against its official hooks documentation) that exposes a pre-submit `UserPromptSubmit` seam.

This is **not a gap:**

- On every other agent the free-form form still works — LAYER makes the agent recognise and apply the prefix itself (it reads the recognition instruction from its context carrier).
- The generated **slash command** (`/background` , form) is the mechanical convenience on the agents that support generated commands (Gemini, Qwen, Codex).

So: **form (free-form) works everywhere via LAYER ; mechanical rewrite is a Claude-Code upgrade; the slash command is the convenience elsewhere.** The slash generators are a nicety, not the load-bearing path.

. Quick-start — how it loads out-of-the-box

You do not install anything per-prompt. Once a project includes the constitution submodule, the system is available:

1. **LAYER is automatic.** The recognition instruction lives in the project's `CLAUDE.md` / `AGENTS.md` / `QWEN.md` / `GEMINI.md` (propagated per § . .). Every agent loads its carrier with every prompt, so it already knows the prefix grammar and the `BACKGROUND` expansion — even without registry-file access (the `BACKGROUND` expansion is inlined into the carrier block).
2. **LAYER is wired by the installer.** Running the project setup invokes `scripts/install_action_prefix.sh` (by reference, per § . .), which:
   - merges the Claude Code `UserPromptSubmit` hook entry into the project's `.claude/settings.json` (idempotent — no duplicate on re-run, pre-existing hooks preserved, `settings.json` backed up first), and
   - runs the slash-command generator so `/background` exists for the other agents.
3. **Use it.** Start a prompt with `BACKGROUND :: <your task>` (form) on any agent, or `/background <your task>` (form) on Gemini/Qwen/Codex.

To verify the system is live, see the Manual §"Troubleshooting".

. What is implemented (read before relying on a form)

Per § . . this is stated as fact, not implied:

Implemented + verified working (per the Implementation Report):

- Form — `ACTION_NAME :: <rest>` — end-to-end: registry → expander library → LAYER- recognition instruction → LAYER- Claude Code hook.
- Form — `PREFIX::ACTION_NAME :: <rest>` (e.g. `DEFAULT::BACKGROUND :: …`) — namespaced `::` form (grammar suite /).
- Form — `/ACTION_NAME <rest>` — the generated bare slash command (`/background`) for Gemini (`.toml`), Qwen (`.toml`), Codex (`prompts/`).
- Form — `/PREFIX::ACTION_NAME <rest>` (e.g. `/DEFAULT::BACKGROUND …`) — namespaced slash form, always-unambiguous.
- Form — `ACTION_NAME ---> <rest>` (e.g. `BACKGROUND ---> …`, and the namespaced `DEFAULT::BACKGROUND ---> …`) — the arrow form, the `--->` body separator recognised by the grammar + LAYER- hook (grammar suite / , paired-mutation meta-test /).
- The `DEFAULT` namespace and the registry keys that drive forms / / (`grammar.default_namespace`, `grammar.namespace_separator`, `grammar.slash_prefix`, `grammar.arrow_form_regex`, `grammar.arrow_body_separator`, per-action `namespaces`, `slash_bare`, `slash_conflicts`), the conflict-aware bare-slash rule (`slash_bare: auto`), and the `/default::background` namespaced slash artefact.
- The `BACKGROUND` and `REMINDER` actions with their verbatim expansions.
- The escape (`\`), stacked-prefix (outer-to-inner), unknown-token-ask, and no-op rules of § .

All five forms are verified end-to-end (grammar suite / , paired-mutation meta-test /). The registry declares the full namespaced grammar (including the arrow

`arrow_form_regex` + `--->` body separator) and the expander library's `apx_parse_prefix` recognises all five forms; there is no remaining spec-only gap in the grammar.

. Related documents

- MANUAL.md — developer/maintainer manual (architecture, registry schema, adding a new action, conflict rule, troubleshooting).
- DIAGRAMS.md — Mermaid diagrams of the expansion flow, the two-layer architecture, the -form grammar decision tree, and the add-a-new-action sequence.
- Source design: `docs/research/action_prefix_system/{RESEARCH.md, DESIGN.md, GRAMMAR_ADDENDUM.md, IMPLEMENTATION_REPORT.md, RULE_DRAFT.md}`.
- Canonical authority: `Constitution.md` § . . .